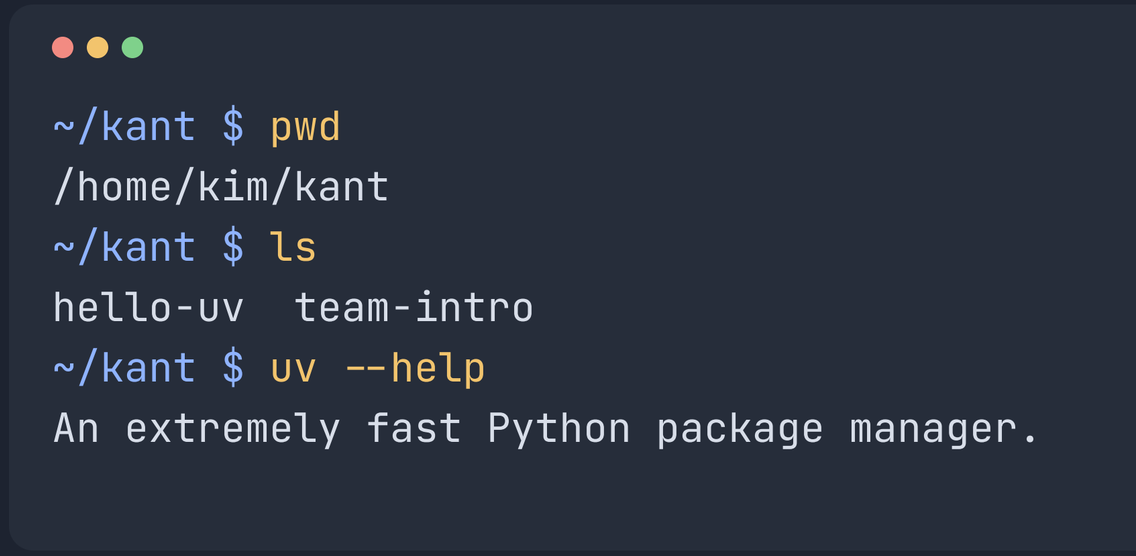
KANT AX/AI 엔지니어 트랙 · 번외

터미널 환경에 적응하기

지금 위치를 확인하는 습관과
-h, --help로 도움 받기



오늘의 순서

1 **탐색기와 터미널 맞춰 보기** — 4–12쪽
더블클릭, 위로, 뒤로, 홈과 cd

2 **위치가 맞아야 명령이 동작해요** — 13–19쪽
같은 명령 다른 결과, 확인 습관, git status

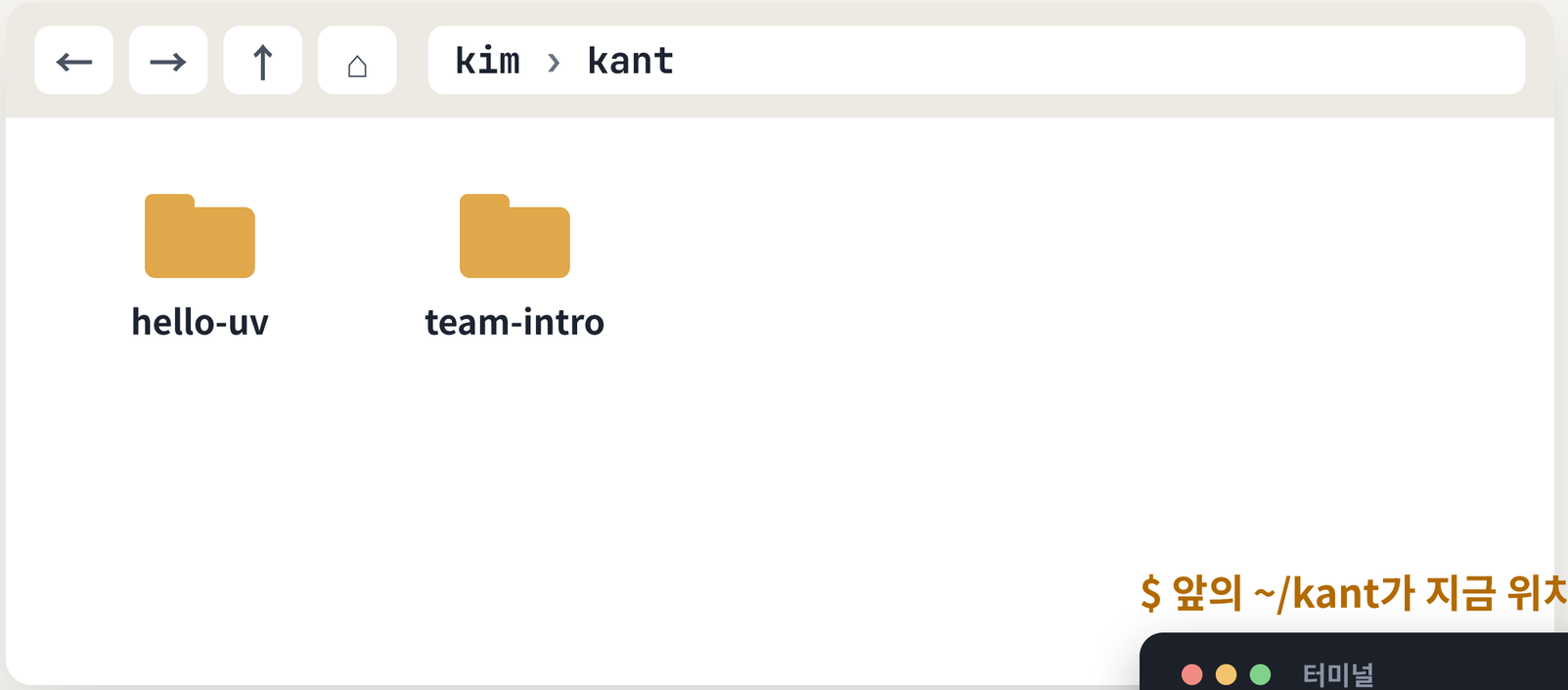
3 **이름에 담긴 관례** — 20–25쪽
명령 이름, -h와 --help, origin과 main

4 **도움말로 스스로 찾기** — 26–34쪽
Usage 읽기, git과 uv 도움말, q로 나오기

3쪽은 들어가기, 35–36쪽은 막힐 때 보는 표와 한 장 요약이에요.

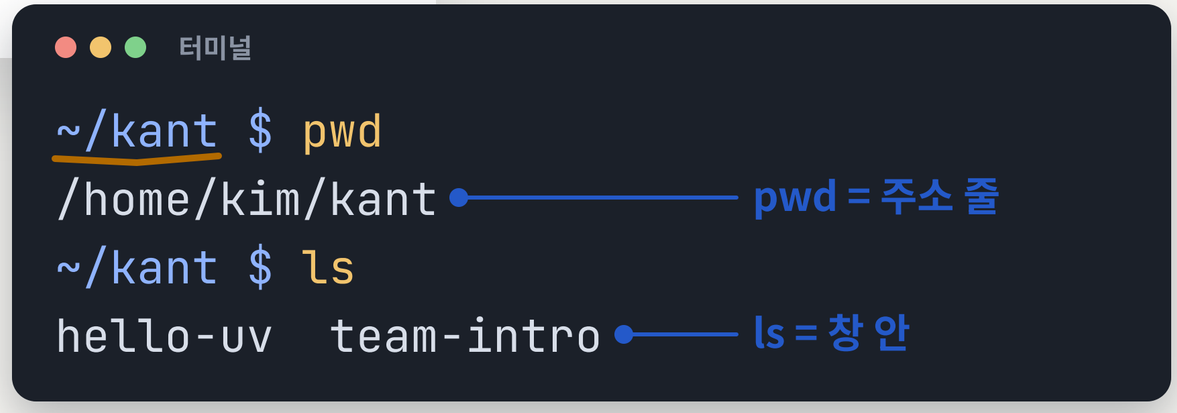
들어가기

탐색기는 그림으로, 터미널은 글자로 위치를 알려 줘요

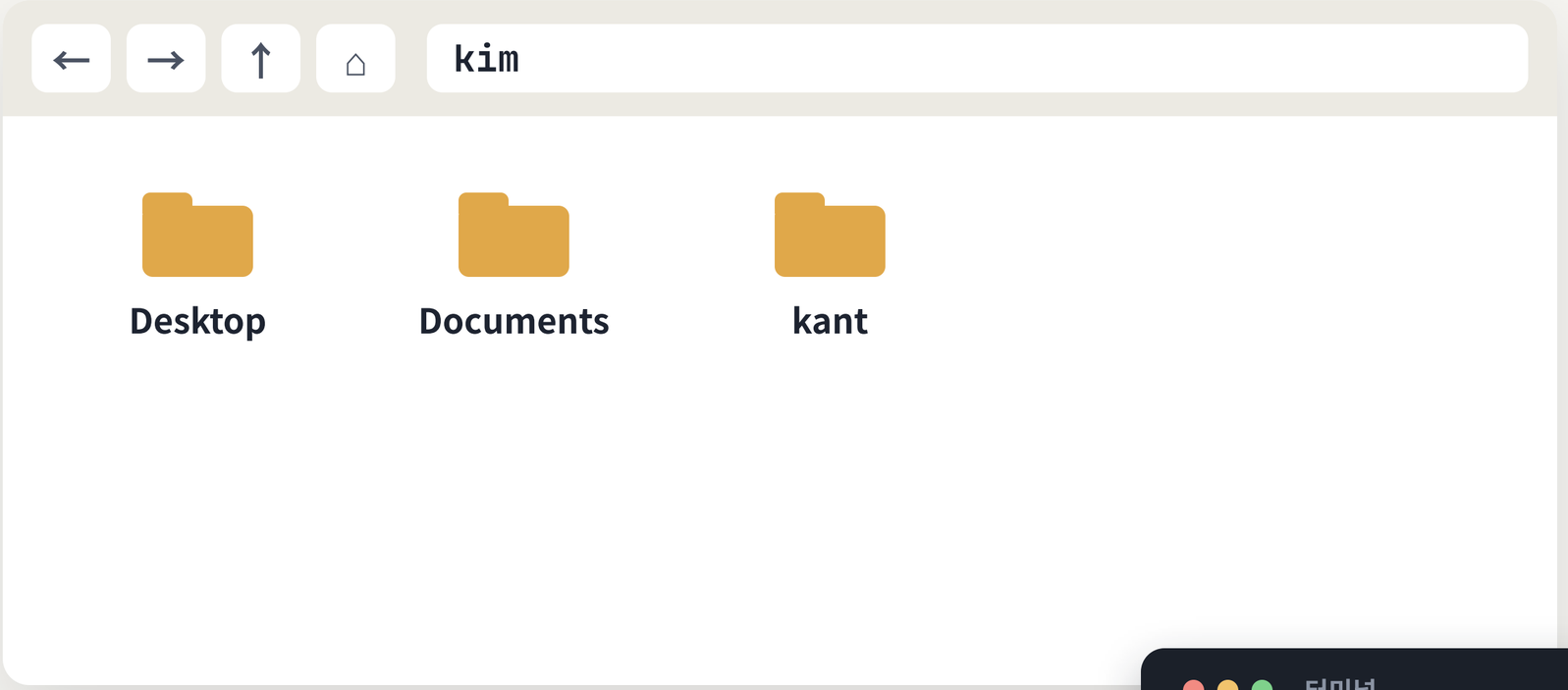


둘 다 kant 폴더 안이에요.
터미널은 물어봐야 알려 줘요.

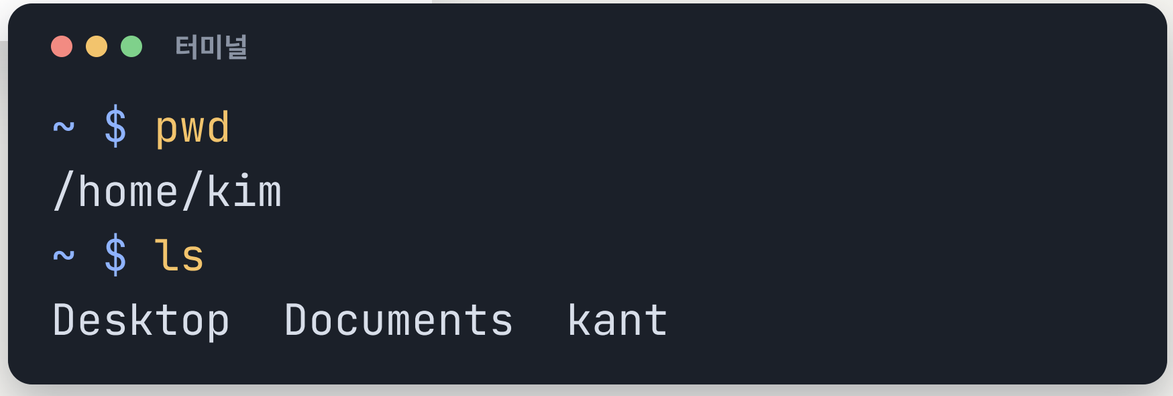
$ 앞의 ~/kant가 지금 위치예요



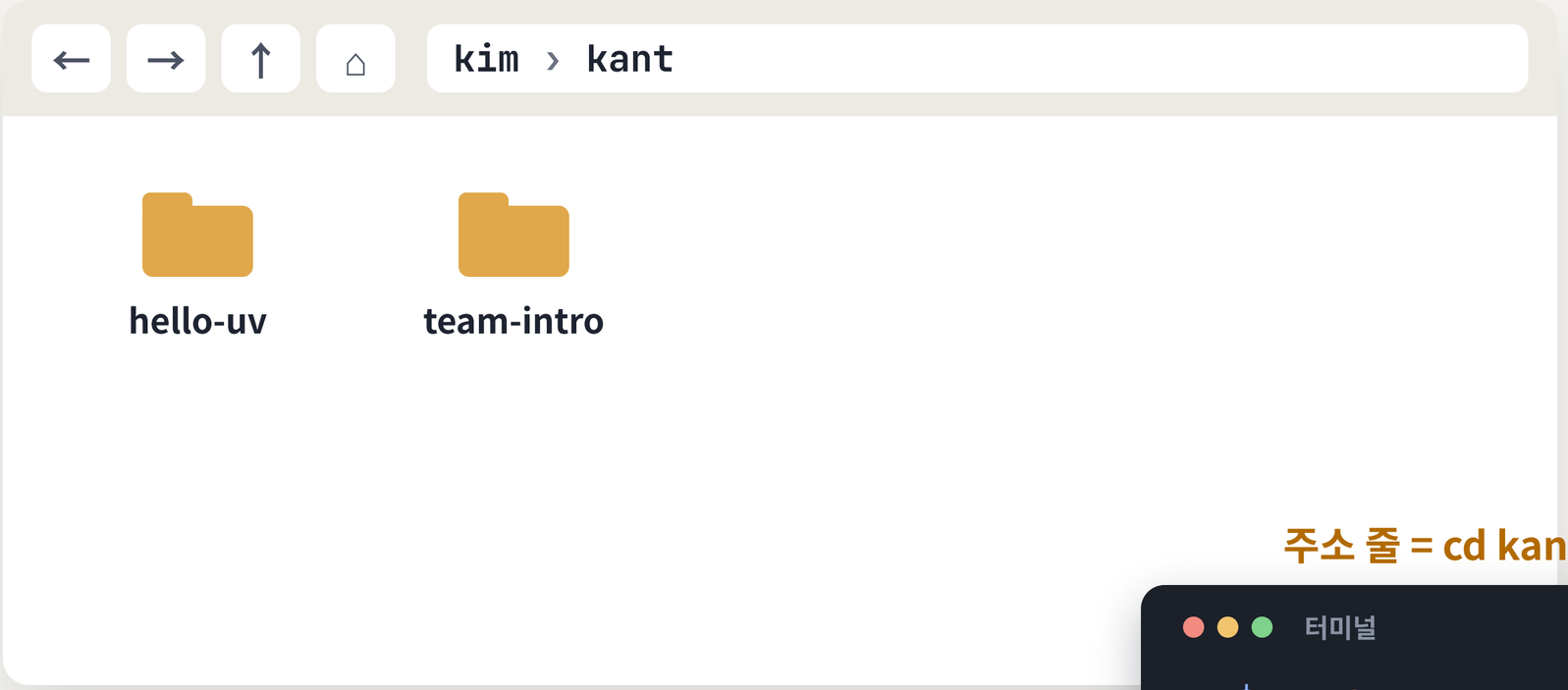
터미널을 열면 홈 폴더에서 시작해요



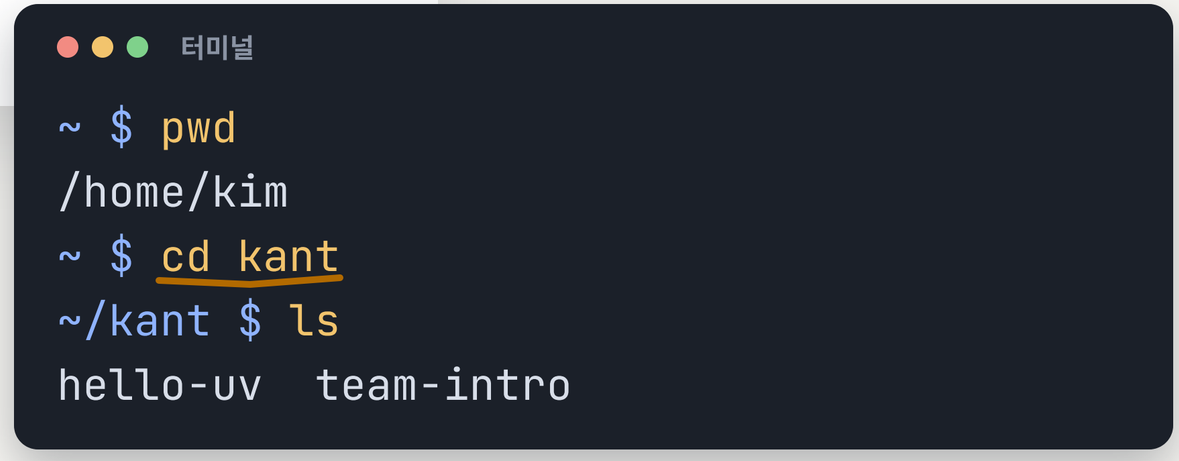
탐색기에서 홈 폴더 창을 연 것과 같아요.



폴더 더블클릭은 cd kant예요

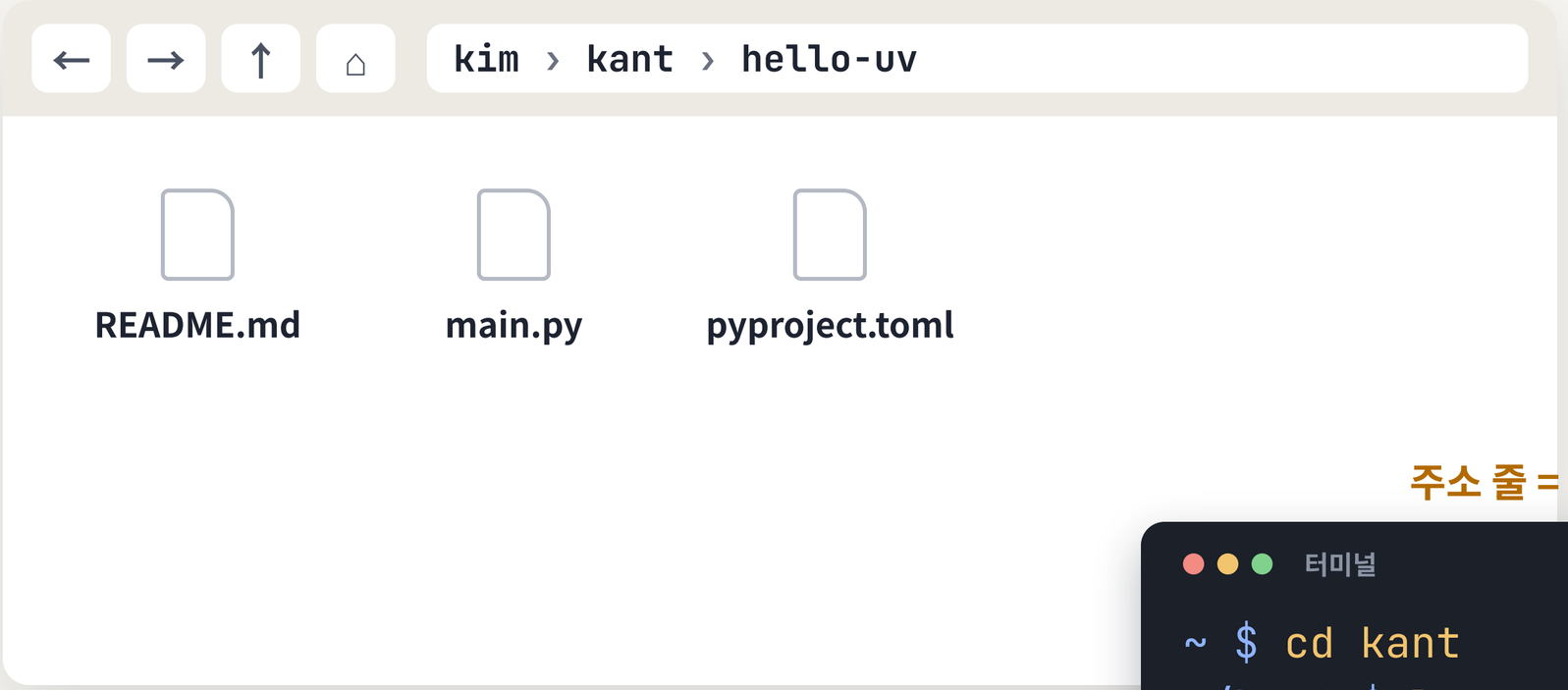


주소 줄 = cd kant



프롬프트가 ~에서 ~/kant로 바뀌었어요. 주소 줄도 같이 바뀌었어요.

한 칸 더 들어가기도 cd예요



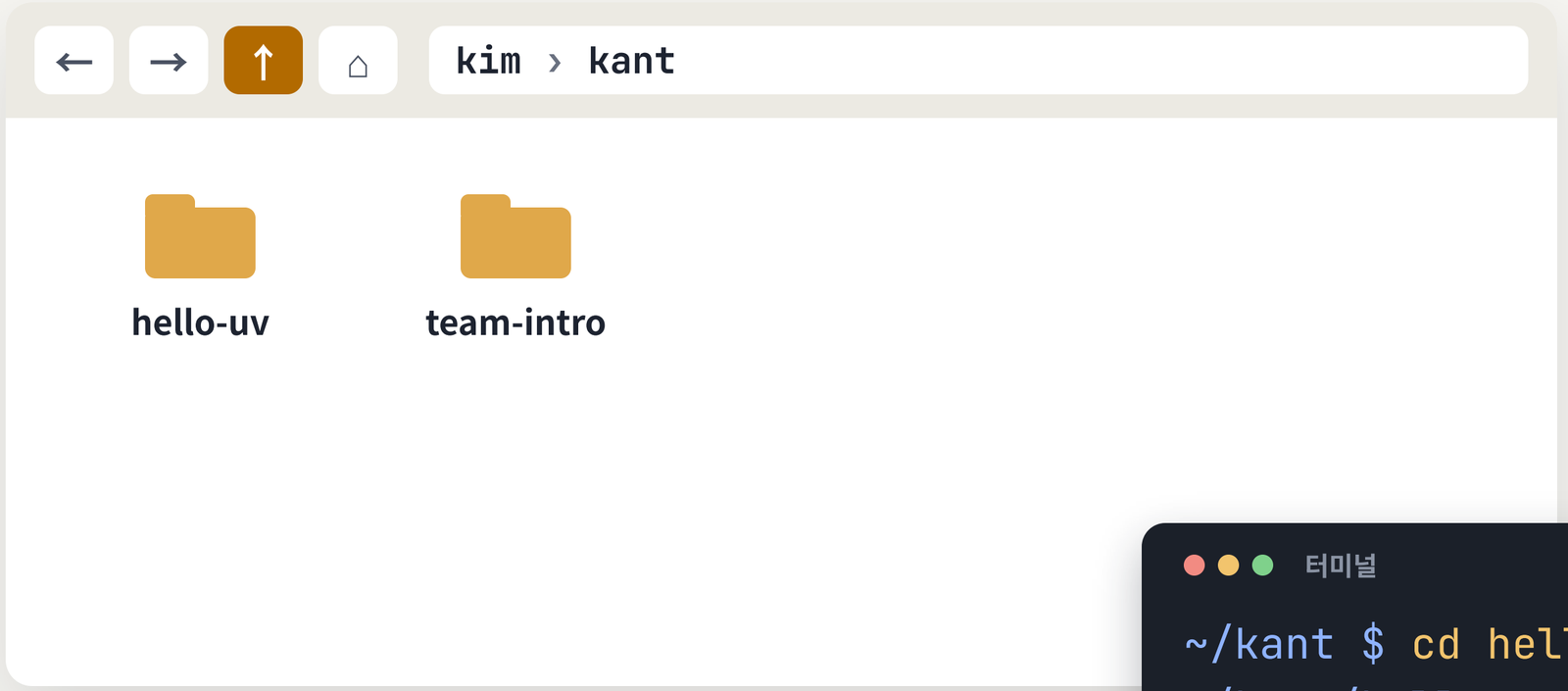
주소 줄 = cd hello-uv

```
터미널
~ $ cd kant
~/kant $ ls
hello-uv  team-intro
~/kant $ cd hello-uv
~/kant/hello-uv $ ls
README.md  main.py  pyproject.toml
```

주소 줄 kim › kant › hello-uv는
/home/kim/kant/hello-uv와 같은 길이에요.

1. 탐색기와 터미널 맞춰 보기

위 폴더(↑)는 cd ..이에요



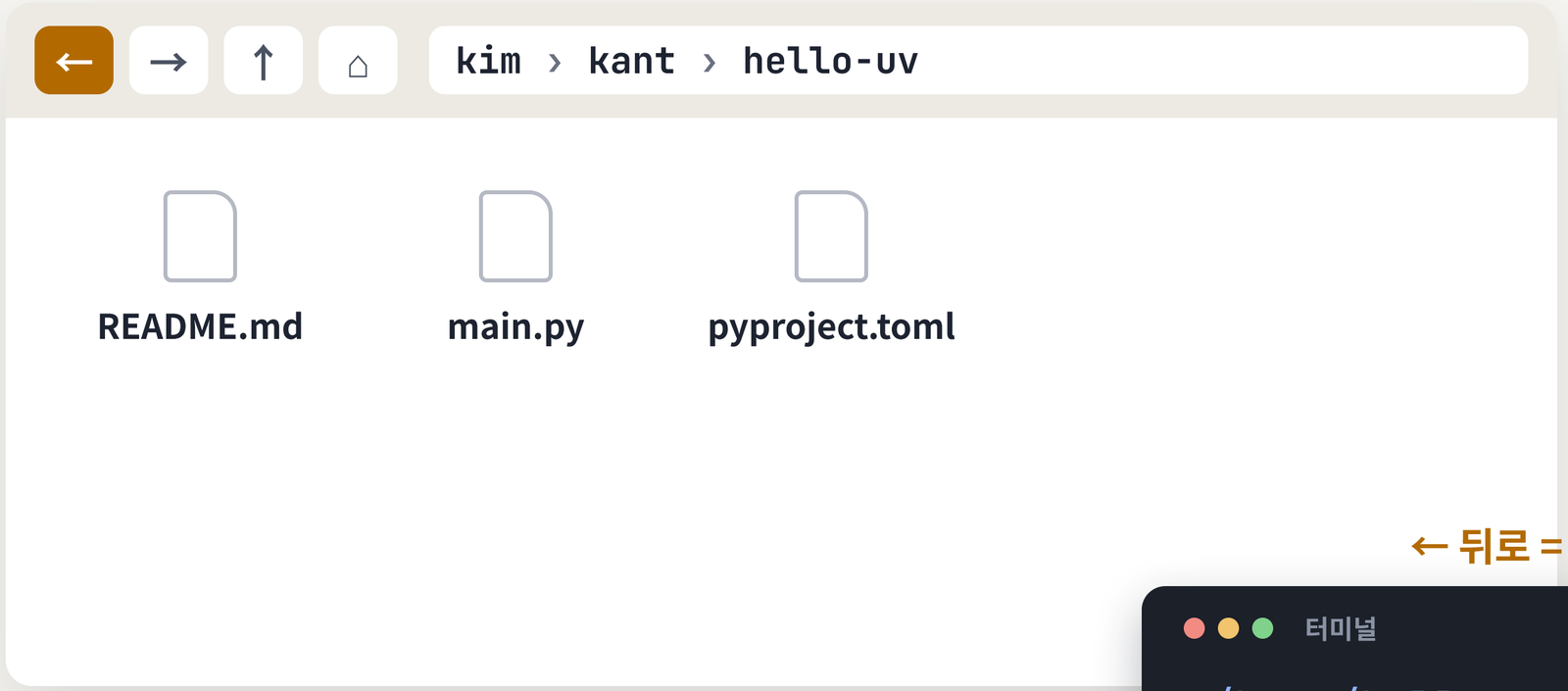
↑ 위 폴더 = cd ..

```
터미널
~/kant $ cd hello-uv
~/kant/hello-uv $ ls
README.md  main.py  pyproject.toml
~/kant/hello-uv $ cd ..
~/kant $ pwd
/home/kim/kant
```

..은 한 칸 위 폴더예요. 01에서 본 기호예요.

1. 탐색기와 터미널 맞춰 보기

뒤로(←)는 cd -예요

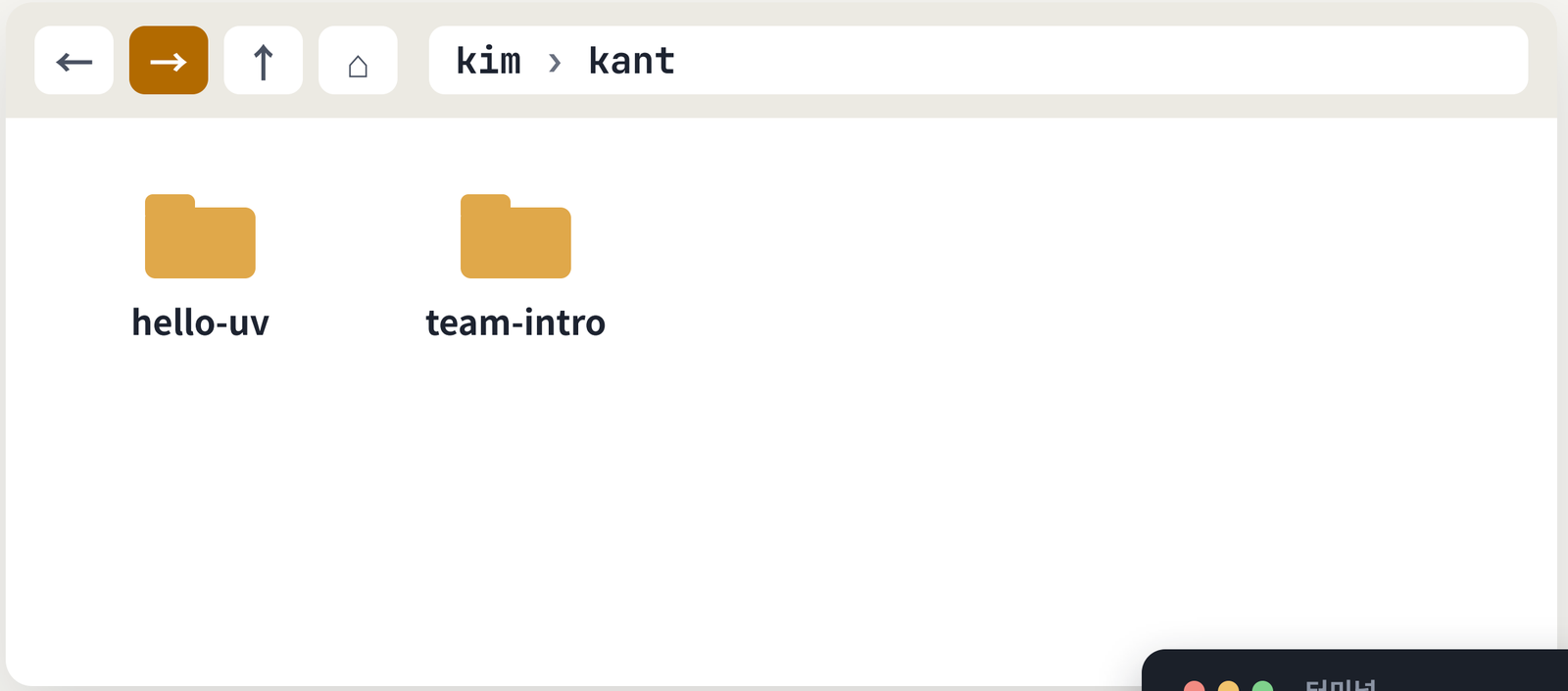


← 뒤로 = cd -

```
터미널
~/kant/hello-uv $ cd ..
~/kant $ pwd
/home/kim/kant
~/kant $ cd -
/home/kim/kant/hello-uv
```

cd -는 바로 전 폴더로 돌아가고, 간 곳을 한 줄 알려 줘요.

앞으로(→)는 cd -를 한 번 더 쳐요



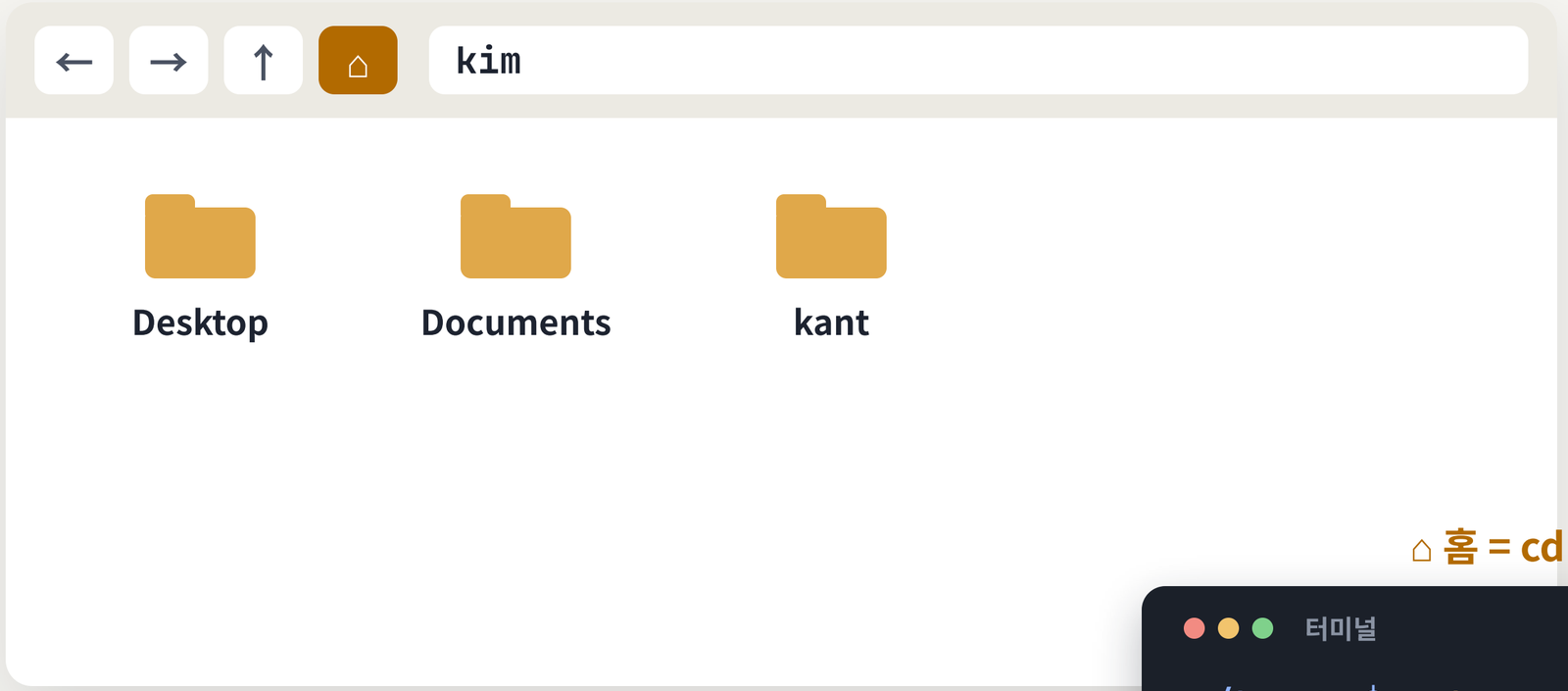
→ 앞으로 = cd -

```
터미널
~/kant $ cd -
/home/kim/kant/hello-uv
~/kant/hello-uv $ cd -
/home/kim/kant
```

cd -는 바로 전 한 곳만 기억해요. 다시 치면 두 폴더를 오가요.

1. 탐색기와 터미널 맞춰 보기

홈(⌂)은 cd ~예요

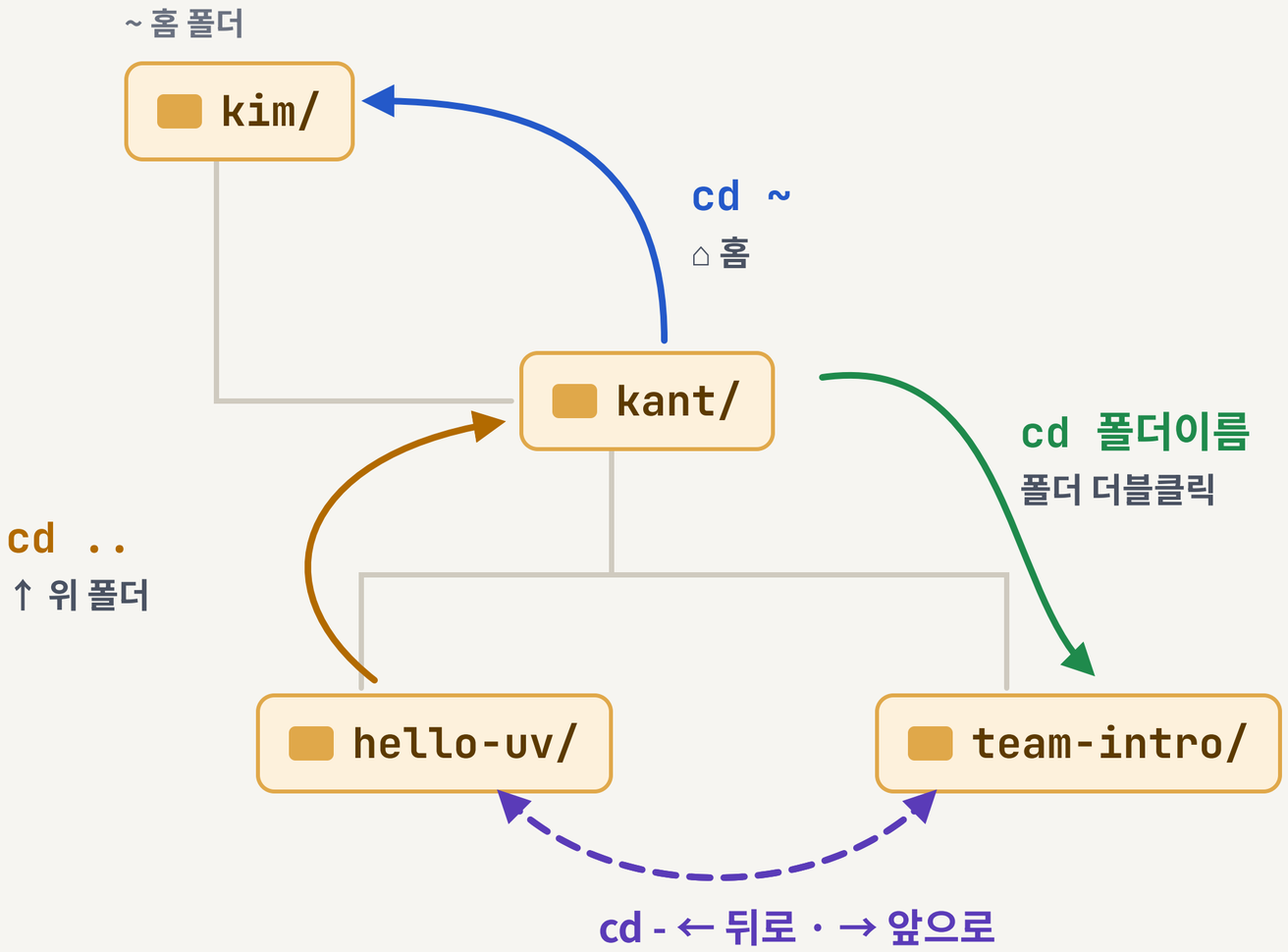


⌂ 홈 = cd ~

```
터미널
~/kant $ cd -
/home/kim/kant
~/kant $ cd ~
~ $ pwd
/home/kim
```

어디에 있든 cd ~ 한 번이면 홈 폴더로 돌아와요.

탐색기 버튼과 cd를 폴더 지도 한 장에 모아요



보는 명령

pwd
주소 줄 · 지금 위치 전체

ls
창 안 · 이 폴더에 든 것

터미널은 창이 없어서
옮길 때마다 이 둘로 확인해요

1. 탐색기와 터미널 맞춰 보기

탐색기와 터미널을 같은 폴더에서 열어요

터미널에서 지금 폴더를 탐색기로

Mac

```
~/kant $ open .
```

Windows Git Bash

```
~/kant $ explorer .
```

점(.)은 지금 폴더예요

탐색기에서 터미널로	Mac	폴더 오른쪽 클릭 › 서비스 › 폴더에서 새로운 터미널 열기
	Windows	폴더 빈 곳 오른쪽 클릭 › Open Git Bash here
VS Code에서	Windows / Mac	폴더를 열고 Ctrl+` / control+` · 그 폴더에서 터미널이 시작돼요

둘을 나란히 띄우고, 터미널에서 만든 파일이 탐색기에 생기는지 봐요.

2. 위치가 맞아야 명령이 동작해요

같은 uv run main.py도 폴더에 따라 결과가 달라요

kant에서 · main.py가 없어요

```
~/kant $ ls
hello-uv  team-intro
~/kant $ uv run main.py
error: Failed to spawn: `main.py`
  Caused by: No such file or
  directory (os error 2)
```

hello-uv에서 · main.py가 있어요

```
~/kant/hello-uv $ ls
README.md  main.py
pyproject.toml
~/kant/hello-uv $ uv run main.py
Hello from hello-uv!
```

명령 뒤의 파일 이름은 지금 폴더에서 찾아요.

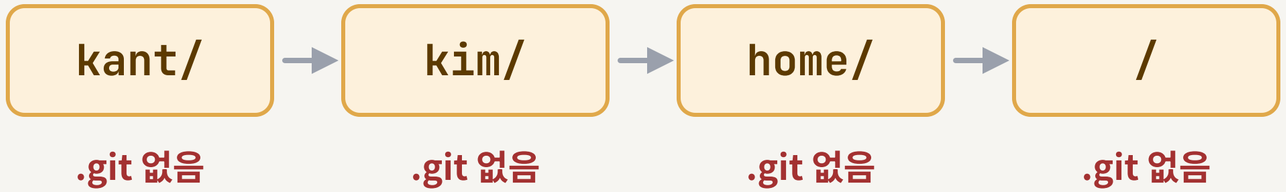
No such file or directory가 보이면 명령을 고치기 전에 pwd와 ls부터 쳐요.

2. 위치가 맞아야 명령이 동작해요

git status도 저장소 폴더 안에서만 동작해요

```
~/kant $ git status
fatal: not a git repository
(or any of the parent directories): .git
```

git이 .git 폴더를 찾아 위로 올라가는 길



parent directories는 이 위 폴더들이에요. 끝까지 없으면 fatal이 나요.

team-intro에서

```
~/kant/team-intro $ git status
On branch main
nothing to commit,
working tree clean
```

06에서 clone한 폴더 안에는
숨김 폴더 .git이 있어요.

2. 위치가 맞아야 명령이 동작해요

홈 폴더에서 git init을 치면 홈 전체가 저장소가 돼요

```
~ $ pwd
/home/kim
~ $ git init
Initialized empty Git repository in /home/kim/.git/
~ $ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitconfig
    Desktop/
    Documents/
    kant/
```

.git이 홈 폴더에 생겼어요

Desktop, Documents까지 모두 git이 지켜봐요

홈 폴더 프롬프트에 (main)이 보이거나 VS Code에 바뀐 파일이 수천 개 뜨면 의심해요.

2. 위치가 맞아야 명령이 동작해요

홈 폴더의 .git만 지우면 돌아와요

```
1  ~ $ pwd
   /home/kim
2  ~ $ ls -a
   .   ..   .git   .gitconfig   Desktop   Documents   kant
3  ~ $ rm -rf .git
4  ~ $ git status
   fatal: not a git repository
   (or any of the parent directories): .git
```

홈 폴더인지 먼저 확인해요

숨김 폴더 .git이 보여요

홈 폴더의 .git 하나만 지워요

이제 홈 폴더는 저장소가 아니에요

rm -rf 뒤에는 꼭 .git만 적어요. 점 하나라도 빠지면 다른 것이 지워져요.

2. 위치가 맞아야 명령이 동작해요

치기 전에 보고, 친 뒤에 다시 봐요



명령이 바꾼 것은 화면에 안 보여요.

cd, mkdir, git add는 성공해도 아무 말이 없어요. 그래서 4번에서 한 번 더 봐요.

2. 위치가 맞아야 명령이 동작해요

git status는 네 가지를 알려 줘요

```
~/kant/team-intro $ echo "# kim" > kim.md
~/kant/team-intro $ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    kim.md

nothing added to commit but untracked files present
(use "git add" to track)
```

1. 지금 이름표 · main 브랜치
2. GitHub과 비교 · 마지막 fetch 기준
3. 바뀐 파일 · kim.md는 아직 Git이 몰라요
4. 다음 할 일 · git add 힌트

모르겠으면 git status. 지금 상태와 다음에 칠 명령을 같이 알려 줘요.

2. 위치가 맞아야 명령이 동작해요

프롬프트에서 위치를 읽어요

Mac 터미널 (zsh)

```
kim@MacBook-Air hello-uv %
```

나@컴퓨터 — 지금 폴더 이름만 — 입력 자리

폴더 이름만 보여요. 전체 경로는 pwd로 봐요.

Windows Git Bash

```
kim@DESKTOP MINGW64 ~/kant/team-intro (main)
$
```

홈에서부터 경로 — 저장소 안이면 이름표

(main)이 보이면 git 명령이 동작하는 폴더예요.

3. 이름에 담긴 관례

명령 이름은 영어 단어를 줄인 거예요

pwd	print working directory 지금 폴더 출력	**mv**	move 옮기기
ls	list 목록	**cp**	copy 복사
cd	change directory 폴더 바꾸기	**rm**	remove 지우기
mkdir	make directory 폴더 만들기	**cat**	concatenate 이어 붙여 보여 주기

뜻을 알면 외우지 않아도 떠올라요. 처음 보는 명령도 뜻부터 찾아봐요.

3. 이름에 담긴 관례

대시 하나는 한 글자, 대시 둘은 단어예요

짧은 이름	긴 이름	--help에 적힌 줄
`ls -a`	`ls --all`	`-a, --all`
`uv -h`	`uv --help`	`-h, --help`
`git commit -m`	`git commit --message`	`-m, --[no-]message`

한 글자 옵션은 붙여 써요

`ls -a -l` = `ls -al`

01에서 쓴 ls -al이 이 모양이에요. 긴 이름은 붙일 수 없어요.

3. 이름에 담긴 관례

여러 도구에서 자주 만나는 옵션이 있어요

`-h, --help`	help	도움말 보기
`--version`	version	버전 보기
`-m`	message	메시지 적기 `git commit -m`
`-r`	recursive	폴더 안까지 전부 `rm -r`

`-f`	force	묻지 않고 강제로 `rm -rf`
`-a`	all	숨긴 것까지 전부 `ls -a`
`-q`	quiet	출력 줄이기
`-v`	verbose	출력 자세히

처음 보는 도구도 --help와 --version은 거의 다 통해요.

대부분 이 뜻이지만 도구마다 다를 수 있어요. 다음 장에서 예외를 봐요.

3. 이름에 담긴 관례

관례는 약속일 뿐이라 예외가 있어요

```
~/kant/hello-uv $ ls -h
README.md  pyproject.toml
main.py    uv.lock
~ $ git -v
git version 2.43.0
~ $ uv -V
uv 0.11.32 (x86_64-unknown-linux-gnu)
```

ls -h는 help가 아니에요

크기를 읽기 쉽게 보여 주는 옵션이에요

버전 글자가 달라요

git은 -v, uv는 대문자 -V

Mac 기본 ls는 --help를 몰라요. man ls로 설명서를 열고 q로 나와요.

헷갈리면 긴 이름 --help부터 쳐 봐요. 가장 많이 통해요.

3. 이름에 담긴 관례

origin과 main도 관례로 붙인 이름이에요

```
~/kant/team-intro $ git remote -v
origin    https://github.com/kim/team-intro.git (fetch)
origin    https://github.com/kim/team-intro.git (push)
~/kant/team-intro $ git branch
* main
```

바꿀 수 있는 이름이지만 모두 같은 이름을 써요.

그래서 검색한 글, 에러 메시지, 팀원의 명령을 그대로 따라 칠 수 있어요.

origin

원래(origin) 가져온 곳

02 · git remote add origin으로 직접
06 · clone하면 git이 붙여 줘요

main

주된 줄기, 첫 브랜치 이름

02 · init.defaultBranch main
예전 기본 이름은 master

3. 이름에 담긴 관례

git push origin main을 도움말로 읽어 봐요

git push -h 첫 줄

```
usage: git push [<options>] [<repository> [<refspec>...]]
```



도움말의 <repository> 자리에 origin, <refspec> 자리에 main을 넣었어요. git pull origin main도 같은 순서예요.

우리 과정은 이 두 칸을 매번 끝까지 적어요.

4. 도움말로 스스로 찾기

Usage 줄의 기호 네 가지를 읽어요

ls --help 첫 줄

```
Usage: ls [OPTION]... [FILE]...
```

[ ] 넣어도 되고 빼도 돼요

... 여러 개 적을 수 있어요

uv add -h 첫 줄

```
Usage: uv add [OPTIONS] <PACKAGES|--requirements <REQUIREMENTS>>
```

< > 내 값으로 바꿔 적어요 · 꼭 넣어요

| 둘 중 하나만 골라요

이렇게 쳐요 `ls` `ls -a` `ls -a -l notes` `uv add fastapi`

4. 도움말로 스스로 찾기

옵션 목록은 짧은 이름, 긴 이름, 설명 순서예요

```
~ $ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
…
```



01에서 쓴 ls -a가 목록 맨 위에 있어요.

4. 도움말로 스스로 찾기

git은 도움말을 세 가지로 보여 줘요

명령	보여 주는 것	이럴 때	열리는 곳
`git --help`	**자주 쓰는 명령 목록**	명령 이름이 기억나지 않을 때	터미널에 바로
`git 명령 -h`	**그 명령의 짧은 옵션표**	옵션 글자가 기억나지 않을 때	터미널에 바로
`git 명령 --help`	**그 명령의 설명서 전체**	자세한 설명과 예시가 필요할 때	Mac은 설명서 화면 Windows는 브라우저

예 `git commit -h` 짧게 `git commit --help` 자세히 `git help commit` --help와 같아요

git에서는 -h와 --help가 달라요. 처음엔 -h부터 쳐요.

4. 도움말로 스스로 찾기

git --help는 명령 이름이 기억나지 않을 때 봐요

```
~ $ git --help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           …

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone     Clone a repository into a new directory
   init      Create an empty Git repository or reinitialize an existing one

…

collaborate (see also: git help workflows)
   fetch     Download objects and refs from another repository
   pull      Fetch from and integrate with another repository or a local branch
   push      Update remote refs along with associated objects
```

묶음 제목 · 하는 일별로

왼쪽 칸 · 명령 이름

오른쪽 칸 · 한 줄 설명

02, 06에서 쓴 clone, init, fetch, pull, push가 다 있어요.

4. 도움말로 스스로 찾기

git commit -h는 옵션 글자가 기억나지 않을 때 봐요

```
~/kant/team-intro $ git commit -h
usage: git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>] [--amend]
                  …

    -q, --[no-]quiet      suppress summary after successful commit
    -v, --[no-]verbose    show diff in commit message template

Commit message options
    -F, --[no-]file <file>
                          read message from file
    --[no-]author <author>
                          override author for commit
    --[no-]date <date>    override date for commit
    -m, --[no-]message <message>
                          commit message

    …
```

목록이 70줄쯤 돼요.
지금 찾는 한 줄만 봐요.

찾던 줄

`-m <message>`

<message>가 붙어 있으니
메시지를 꼭 적어요

4. 도움말로 스스로 찾기

설명서 화면이 열리면 q로 나와요

```
GIT-COMMIT(1)                  Git Manual                  GIT-COMMIT(1)

NAME
       git-commit - Record changes to the repository

SYNOPSIS
       git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>]
                  …

DESCRIPTION
       Create a new commit containing the current contents of the
       index and the given log message describing the changes. …

 Manual page git-commit(1) line 1 (press h for help or q to quit)
```

q	나가기
Space	다음 쪽
b	앞 쪽
/단어	단어 찾기

git log가 길 때도 이 화면이 나와요.
맨 아래에 : 이나 (END)가 보이면 q예요.

4. 도움말로 스스로 찾기

uv --help로 uv 명령 목록을 봐요

```
~/kant/hello-uv $ uv --help
An extremely fast Python package manager.

Usage: uv [OPTIONS] <COMMAND>

Commands:
  auth        Manage authentication
  run         Run a command or script
  init        Create a new project
  add         Add dependencies to the project
  remove      Remove dependencies from the project
  version     Read or update the project's version
  sync        Update the project's environment
  lock        Update the project's lockfile
  export      Export the project's lockfile to an alternate format
  tree        Display the project's dependency tree
  …
```

노란 줄

09에서 쓴 명령

`run init add`
`remove sync tree`

---

uv는 -h와 --help가 같아요

더 긴 설명은 help 뒤에 명령 이름

`uv help add`

4. 도움말로 스스로 찾기

오타를 내면 도구가 먼저 알려 줘요

git stauts

```
~/kant/team-intro $ git stauts
git: 'stauts' is not a git command.
See 'git --help'.

The most similar command is
    status
```

uv ad fastapi

```
~/kant/hello-uv $ uv ad fastapi
error: unrecognized subcommand 'ad'

  tip: some similar subcommands
  exist: 'audit', 'add'

Usage: uv [OPTIONS] <COMMAND>

For more information, try '--help'.
```

에러 메시지는 맨 아래 몇 줄을 읽어요. 노란 줄만 읽어도 고칠 수 있어요.

4. 도움말로 스스로 찾기

모르는 기능을 도움말로 찾아가요

하고 싶은 일 · 설치한 패키지를 나무 모양으로 보고 싶어요

1
```
~/kant/hello-uv $ uv --help
  …
  tree        Display the project's dependency tree
  …
```
목록에서 tree를 찾았어요

2
```
~/kant/hello-uv $ uv tree -h
Display the project's dependency tree

Usage: uv tree [OPTIONS]
```
옵션이 다 대괄호라
그냥 쳐도 돼요

3
```
~/kant/hello-uv $ uv tree
Resolved 1 package in 2ms
hello-uv v0.1.0
```
패키지를 넣으면
아래로 가지가 늘어요

목록에서 이름 찾기, -h로 쓰는 법 보기, 쳐 보기. 이 세 단계면 돼요.

정리

터미널에서 막히면 이 표를 봐요

보이는 것	원인	칠 것
`fatal: not a git repository`	git 저장소 폴더 밖이에요	**`pwd` 뒤 `cd 저장소`**
`error: Failed to spawn: `main.py``	main.py가 없는 폴더예요	**`ls` 뒤 `cd hello-uv`**
`No such file or directory`	위치나 이름이 틀렸어요	**`pwd`, `ls`, Tab**
화면이 멈추고 : 이나 (END)만 보여요	설명서 화면이에요	**`q`**
홈 폴더에서 git status가 동작해요	홈에서 git init을 쳤어요	**16쪽 순서**
옵션이 기억나지 않아요	도움말을 봐요	**`명령 -h`, `--help`**

한 장 요약

어디에 있나

`pwd`
지금 위치

`ls`
안에 든 것

`git status`
저장소 상태

옮겨 다니기

`cd 폴더`
들어가기

`cd ..`
한 칸 위로

`cd -`
바로 전 폴더

`cd ~`
홈 폴더

도움 받기

`명령 --help`
도움말

`git 명령 -h`
git 짧은 옵션표

`uv 명령 -h`
uv 도움말

`q`
설명서 화면 나가기

치기 전에 어디인지 보고, 친 뒤에 잘 됐는지 봐요.

대시 하나는 한 글자(-h), 대시 둘은 단어(--help). 헷갈리면 --help.